



Programación Paralela

Proyecto Final

Analizador Paralelo de Archivos de texto

(Recursiva)

Integrantes del Equipo:

Anthony Guzmán - 20231182

Líder del Equipo:

Marializ Disla - 20231098

Fecha de entrega:

22-08-2026

Índice

1. Introducción	3
Presentación general del proyecto	3
Justificación del tema elegido	3
Objetivos	3
Objetivo general:.....	3
Objetivos específicos:	3
2. Descripción del Problema	4
Contexto del problema	4
Aplicación en un escenario real	4
Importancia del paralelismo y el alto procesamiento	4
3. Cumplimiento de los Requisitos del Proyecto	4
4. Diseño de la Solución	5
Arquitectura general del sistema	5
Estrategia de paralelización utilizada	6
Herramientas y tecnologías empleadas	6
5. Implementación Técnica.....	6
Explicación del proyecto.....	6
Uso de mecanismos de sincronización	7
Justificación técnica de las decisiones	7
6. Evaluación de Desempeño.....	7
Análisis de cuellos de botella y limitaciones	8
7. Trabajo en Equipo	8
Reparto de tareas	8
Herramientas de coordinación:	8
9. Referencias	9
10. Anexos.....	9
Manual de ejecución:.....	9

1. Introducción

Presentación general del proyecto

Este proyecto analiza automáticamente todos los archivos de texto de una carpeta y sus subcarpetas para obtener estadísticas: cantidad de archivos, líneas, palabras, caracteres, la frecuencia de cada palabra y las coincidencias de una lista de términos clave. Los documentos contienen texto real de libros clásicos de dominio público. El programa recorre la carpeta de forma recursiva para encontrar los archivos y luego los analiza usando programación paralela, repartiendo el trabajo entre los núcleos del procesador. Al final compara el rendimiento de una versión secuencial con dos versiones paralelas

Justificación del tema elegido

El problema encaja perfectamente con el paralelismo porque cada archivo se puede analizar de forma independiente de los demás, así que varios archivos se procesan al mismo tiempo. Además, recorrer una estructura de carpetas y subcarpetas es un recorrido recursivo natural. Elegimos este tema porque permite trabajar con un gran volumen de datos (miles de archivos) y con un procesamiento intensivo (contar y clasificar millones de palabras), que es donde el paralelismo muestra una ventaja clara sobre el modo secuencial.

Objetivos

Objetivo general:

Demostrar la ventaja del paralelismo analizando un gran volumen de archivos de texto, manejando correctamente la sincronización del reporte compartido.

Objetivos específicos:

- ✚ Implementar una versión secuencial y dos versiones paralelas del análisis.
- ✚ Medir y comparar tiempo, speedup y eficiencia de cada estrategia.
- ✚ Compartir un reporte general entre tareas usando lock para evitar errores.
- ✚ Analizar la escalabilidad y los cuellos de botella de cada estrategia.

2. Descripción del Problema

Contexto del problema

Las empresas almacenan miles de documentos de texto (informes, registros, correos, logs). Revisarlos uno por uno para sacar estadísticas es lento. El proyecto automatiza esa tarea, el usuario indica una carpeta y el programa recorre todas las subcarpetas, encuentra los archivos de texto, los analiza y genera un reporte único con las estadísticas de todos.

Aplicación en un escenario real

Este tipo de análisis se usa en buscadores e indexadores de documentos, en el análisis de archivos de registro (logs) de servidores y en herramientas de minería de texto. En todos esos casos hay que procesar grandes cantidades de archivos lo más rápido posible, por lo que el paralelismo es la solución natural.

Importancia del paralelismo y el alto procesamiento

Como cada archivo es independiente, se pueden analizar varios a la vez en distintos núcleos. Para que la ventaja sea significativa, el trabajo por archivo debe ser intensivo en CPU y no solo lectura de disco. Los archivos contienen texto real de libros clásicos de dominio público, y el analizador además de contar líneas, palabras y caracteres, busca dentro de cada documento las 800 palabras más frecuentes del corpus (como hace un buscador o un filtro de contenido), recorriendo todo el texto por cada término. Esa búsqueda es un trabajo pesado de CPU que domina sobre la lectura del disco, de modo que repartirlo entre los núcleos reduce el tiempo total de forma notable. Si el análisis fuera solo lectura de disco, el paralelismo apenas ayudaría, porque el disco es un recurso compartido.

3. Cumplimiento de los Requisitos del Proyecto

Ejecución simultánea de múltiples tareas

El sistema procesa varios archivos al mismo tiempo en distintos núcleos del procesador. En la clase Analizador, los archivos se reparten y cada grupo se procesa dentro de una tarea creada con Task.Factory.StartNew, esperando a que todas terminen con Task.WhenAll. Mientras un núcleo analiza un conjunto de documentos, los demás analizan otros conjuntos de forma simultánea. Esto se confirma en las mediciones: el tiempo total baja de forma notable respecto al modo secuencial, lo que solo es posible si el trabajo se ejecuta en paralelo.

Necesidad de compartir datos entre tareas

Todas las tareas contribuyen a un único reporte general (la clase Reporte), que acumula el total de archivos, líneas, palabras, caracteres y coincidencias de términos clave. Como varias tareas intentan actualizar ese mismo objeto a la vez, existe una necesidad real de compartir datos y de protegerlos. Cada tarea calcula su reporte parcial por separado y solo al combinarlo en el reporte general entra en un bloque lock, evitando que dos tareas lo modifiquen al mismo tiempo. Que los tres modos produzcan exactamente el mismo total demuestra que la sincronización es correcta.

Exploración de diferentes estrategias de paralelización

El proyecto no usa una sola forma de paralelizar, sino que compara tres estrategias sobre el mismo problema: secuencial (los archivos uno por uno, como base de comparación), paralelo por archivo (una tarea por cada archivo) y paralelo por grupos (los archivos se dividen en grupos, uno por núcleo, y se crea una tarea por grupo). Comparar las tres permite observar que el paralelo por grupos es más eficiente que el paralelo por archivo, porque crear una tarea por cada archivo genera demasiado costo de coordinación y contención en el lock.

Escalabilidad con más recursos

El sistema aprovecha automáticamente los recursos disponibles: el número de grupos se ajusta a la cantidad de núcleos del procesador mediante `Environment.ProcessorCount`, de modo que en un equipo con más núcleos se crean más tareas simultáneas sin cambiar el código. Además, la cantidad de archivos es configurable, lo que permite medir el comportamiento al aumentar el volumen. Las pruebas de escalabilidad muestran que el paralelismo mantiene su ventaja al crecer los datos, aunque la eficiencia disminuye ligeramente con volúmenes muy grandes por la ley de Amdahl.

Métricas de evaluación del rendimiento

El programa mide el rendimiento de forma cuantitativa. Con `System.Diagnostics.Stopwatch` cronometra cada uno de los tres modos y, a partir de esos tiempos, calcula el speedup (cuántas veces más rápido es el paralelo que el secuencial) y la eficiencia (speedup dividido entre el número de núcleos). Los resultados se muestran en consola y se exportan a un archivo CSV, con el que se generan gráficas comparativas de tiempo, speedup y eficiencia, aportando evidencia objetiva de la ventaja del paralelismo.

Aplicación a un problema del mundo real

El análisis masivo de documentos de texto es una tarea real y común: los buscadores e indexadores, las herramientas de análisis de registros (logs) y los sistemas de minería de texto necesitan procesar grandes cantidades de documentos rápidamente. El proyecto reproduce ese escenario analizando un corpus de libros clásicos de dominio público, extrayendo estadísticas y buscando términos clave dentro de cada documento, tal como lo haría un motor de búsqueda o un filtro de contenido. Por eso el problema es representativo de una necesidad real de la industria.

4. Diseño de la Solución

Arquitectura general del sistema

Reporte: guarda las estadísticas (contadores y el diccionario de frecuencias) y sabe combinarse con otro reporte.

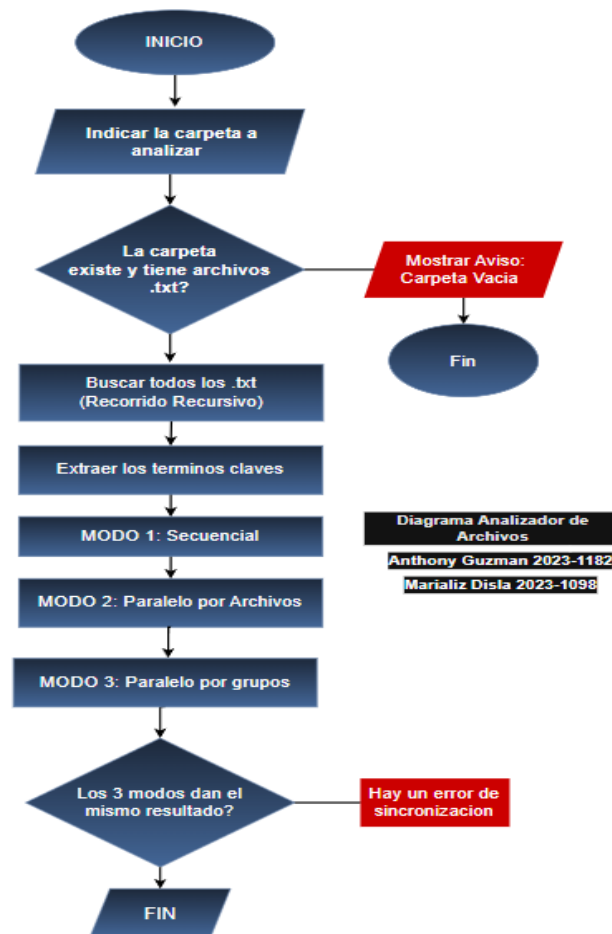
ClavesFrecuentes: obtiene las palabras más frecuentes de los propios documentos, que luego se usan como términos clave en la búsqueda.

BuscadorArchivos: recorre las carpetas de forma recursiva y devuelve la lista de archivos.txt.

Analizador: contiene las tres estrategias de análisis y el manejo del reporte compartido.

Program: este genera el corpus. Ejecuta y cronometra los tres modos y muestra el reporte.

Diagrama de componentes/tareas paralelas



Estrategia de paralelización utilizada

El recorrido de carpetas es recursivo (una carpeta puede contener subcarpetas). El análisis usa paralelismo de datos se toma la lista de archivos y se reparte entre varias tareas. Se comparan dos formas de repartir: una tarea por archivo (muy fina) y una tarea por grupo de archivos (más gruesa), para observar cómo afecta el tamaño de las tareas al rendimiento.

Herramientas y tecnologías empleadas

Lenguaje: C# sobre .NET 10.

Paralelismo: Task Parallel Library (Task.Factory.StartNew y Task.WhenAll) y la instrucción lock para la sincronización.

Medición: System.Diagnostics.Stopwatch.

Graficas: Draw.io y Excel.

Control de versiones: Git / GitHub.

5. Implementación Técnica

Estructura del proyecto

/src - Código Fuente

/tests – Pruebas

/metrics – Resultados de las métricas

/docs – Documentación

Explicación del código clave

El proyecto es una aplicación de consola en C# (.NET 10) que analiza en paralelo todos los archivos de texto de una carpeta y sus subcarpetas, y compara el rendimiento de tres formas distintas de hacerlo. El sistema está organizado en cinco componentes, cada uno con una responsabilidad clara:

Reporte es la estructura de datos que guarda las estadísticas del análisis: el total de archivos, líneas, palabras, caracteres y coincidencias de términos clave, además de un diccionario con la frecuencia de cada palabra. Su método `Combinar` permite sumar dos reportes en uno solo, lo cual es esencial para el paralelismo, ya que cada tarea genera su propio reporte parcial que luego se une al reporte general.

BuscadorArchivos recorre la carpeta indicada de forma recursiva (entra en cada subcarpeta, y en las subcarpetas de esas, hasta el fondo) y devuelve la lista de todos los archivos .txt encontrados. Aquí está la descomposición recursiva del proyecto: el método se llama a sí mismo por cada subcarpeta.

ClavesFrecuentes lee todos los documentos y extrae las 800 palabras más frecuentes del corpus. Esas palabras se convierten en los "términos clave" que después se buscarán dentro de cada archivo. Esto hace que la búsqueda se adapte automáticamente a cualquier conjunto de documentos, sin importar el idioma.

Analizador es el núcleo del programa. Contiene el método que analiza un archivo individual (cuenta sus palabras, líneas y caracteres, y busca en él los términos clave) y las tres estrategias de ejecución: secuencial, paralelo por archivo y paralelo por grupos. También gestiona el reporte compartido y su protección con lock.

Program es el punto de entrada que coordina todo: recibe la ruta de la carpeta a analizar, llama al buscador para obtener los archivos, pide los términos clave, ejecuta y cronometra los tres modos

con Stopwatch, y finalmente muestra el reporte con las estadísticas, los tiempos, el speedup y la eficiencia, guardando además los resultados en un archivo CSV.

El flujo completo de ejecución es el siguiente: el programa recibe una carpeta; si no existe o no contiene archivos .txt, muestra un aviso y termina. Si sí los contiene, BuscadorArchivos obtiene recursivamente la lista de documentos y ClavesFrecuentes extrae los términos clave. Luego se ejecutan los tres modos sobre exactamente los mismos archivos: primero el secuencial, que procesa los documentos uno por uno y sirve como base de comparación; después el paralelo por archivo, que crea una tarea por cada documento; y por último el paralelo por grupos, que divide los archivos en grupos (uno por núcleo del procesador) y crea una tarea por grupo. En los dos modos paralelos, las tareas se crean con Task.Factory.StartNew y el programa espera a que todas terminen con Task.WhenAll; cada tarea trabaja sobre su propio reporte parcial y solo al final lo une al reporte general dentro de un bloque lock, garantizando que no haya conflictos al escribir en el dato compartido.

Al terminar, el programa imprime el reporte general (archivos, líneas, palabras, caracteres, coincidencias y las palabras más repetidas) y una comparativa de rendimiento con el tiempo de cada modo, su speedup respecto al secuencial y su eficiencia. Como paso de verificación, comprueba que los tres modos produzcan exactamente el mismo resultado, lo que confirma que la paralelización es correcta y no introdujo errores de sincronización.

Uso de mecanismos de sincronización

El reporte general es el dato compartido. Cada tarea produce su propio reporte parcial y solo al final lo combina en el general dentro de un bloque lock. El lock es un candado solo una tarea a la vez puede entrar a combinar, y las demás esperan su turno, evitando que dos tareas modifiquen los mismos contadores o el mismo diccionario al mismo tiempo. La verificación al final confirma que los tres modos producen exactamente el mismo reporte.

Justificación técnica de las decisiones

Por grupos en vez de por archivo: crear miles de tareas (una por archivo) tiene un costo alto y obliga a combinar miles de veces bajo lock, agrupar reduce ese costo.

Combinar una sola vez por tarea: cada tarea acumula en un reporte parcial propio (sin lock) y solo entra al lock una vez, reduciendo la espera entre tareas.

Términos clave tomados de los propios documentos: el programa usa las 800 palabras más frecuentes del texto analizado, así la búsqueda se adapta a cualquier carpeta.

Calentamiento (warm-up): se hace una pasada antes de medir para que el disco quede en caché y la medición refleje el trabajo de CPU.

6. Evaluación de Desempeño

Comparativa entre ejecución secuencial y paralela

Se analizaron 400 archivos con texto de libros de dominio público con unos 4 millones de palabras en total, buscando las 800 palabras más frecuentes en cada uno, con cada estrategia, en un equipo con 12 núcleos lógicos en compilación Release. La tabla muestra el tiempo, el speedup respecto al secuencial y la eficiencia (speedup entre número de núcleos).

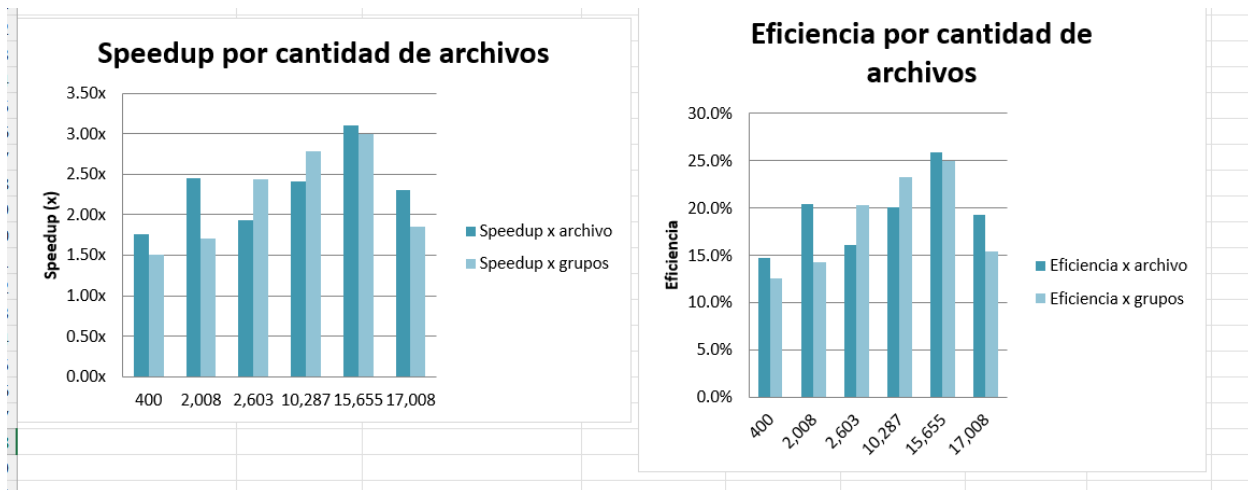
Estrategia	Tiempo (ms)	Speedup	Eficiencia
Secuencial	2267.9	1.00x	100%
Paralelo por archivo	670.2	3.38x	28%
Paralelo por grupos	601.6	3.77x	31%

El analizador funciona con documentos en cualquier idioma, porque los términos clave se extraen de los propios textos. Además del corpus en inglés usado en esta medición, el sistema se probó con un corpus en español de once libros clásicos con más de 2,000 archivos.

Métricas: tiempo de ejecución, eficiencia, escalabilidad

Escalabilidad del Analizador de Archivos							
Núcleos del procesador:	12						
Cantidad de archivos	Secuencial (ms)	Paralelo x archivo (ms)	Paralelo x grupos (ms)	Speedup x archivo	Speedup x grupos	Eficiencia x archivo	Eficiencia x grupos
400	3,543	2,011	2,347	1.76x	1.51x	14.7%	12.6%
2,008	4,840	1,979	2,830	2.45x	1.71x	20.4%	14.3%
2,603	7,602	3,934	3,115	1.93x	2.44x	16.1%	20.3%
10,287	9,412	3,912	3,374	2.41x	2.79x	20.0%	23.2%
15,655	18,807	6,054	6,267	3.11x	3.00x	25.9%	25.0%
17,008	18,890	8,174	10,191	2.31x	1.85x	19.3%	15.4%

Gráficas o tablas con resultados



Análisis de cuellos de botella y limitaciones

La estrategia por archivo crea una tarea por cada archivo (400 tareas) y combina 400 veces dentro del lock; ese costo de coordinación limita un poco su speedup. La estrategia por grupos crea pocas tareas, una por núcleo y combina pocas veces, por lo que en la medición rindió algo mejor. El speedup no llega a ser igual al número de núcleos por varias razones: el procesador tiene 12 núcleos lógicos pero unos 6 físicos, así que el techo real ronda 6x; la construcción del diccionario de frecuencias consume mucha memoria y no escala del todo; y la parte que junta los reportes bajo lock es serial. Aun así, se ve reflejada una ventaja clara del paralelismo. Además, si el análisis fuera solo lectura de disco, el paralelismo apenas ayudaría, porque el disco es un recurso compartido.

7. Trabajo en Equipo

Reparto de tareas

Marializ Disla

Búsqueda recursiva, integración del programa, reportes, rendimiento, verificación, test y Documentación.

Anthony Guzmán

Lógica del analizador, métodos secuencial y paralelo, sincronización, paralelo por grupos, test y Documentación.

Herramientas de coordinación:

Git y GitHub para el repositorio y el código.

Microsoft teams para coordinación, creación y update de cambios.

8. Conclusiones

Principales aprendizajes técnicos:

Los principales aprendizajes diríamos que fueron el paralelismo de datos es decir repartir una lista de archivos entre tareas es sencillo y da una ventaja clara cuando el trabajo por elemento es intensivo en CPU. Compartir un dato entre tareas exige sincronización, el lock protege el reporte general sin equivocar los resultados. También que el tamaño de las tareas importa y que muchas tareas diminutas cuestan más que pocas tareas grandes.

Retos enfrentados y superados

Evitar que el disco fuera el cuello de botella esto se resolvió haciendo el análisis en CPU y calentando la caché antes de medir. Lograr que los tres modos dieran el mismo reporte, se resolvió acumulando en reportes parciales y combinando bajo lock.

Posibles mejoras

Usar `Parallel.ForEach` para un reparto automático de la carga. Usar una estructura concurrente (`ConcurrentDictionary`) para reducir la espera del lock y analizar archivos mucho más grandes o en varios formatos o extensiones.

9. Referencias

O'Reilly Media. (s. f.). Parallel programming with C# and .NET. O'Reilly Learning.

Microsoft. (s. f.). The lock statement - C# reference. Microsoft Learn.

[The lock statement - synchronize access to shared resources - C# reference | Microsoft Learn](#)

10. Anexos

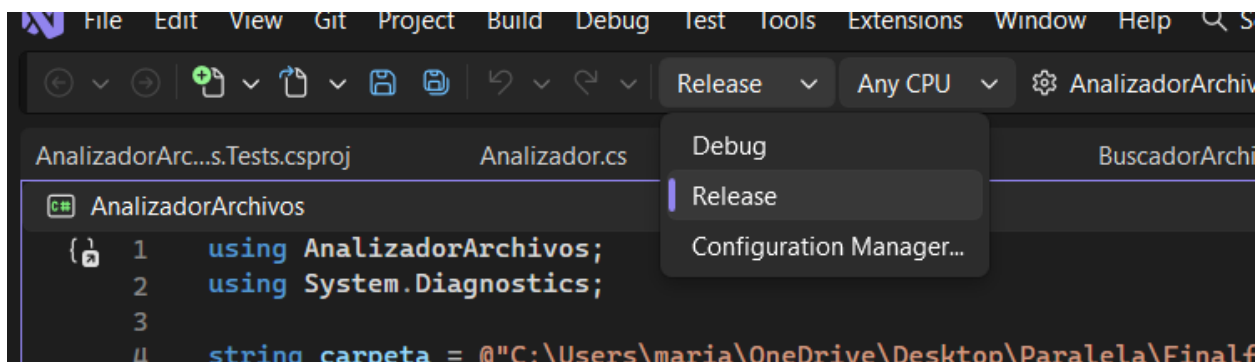
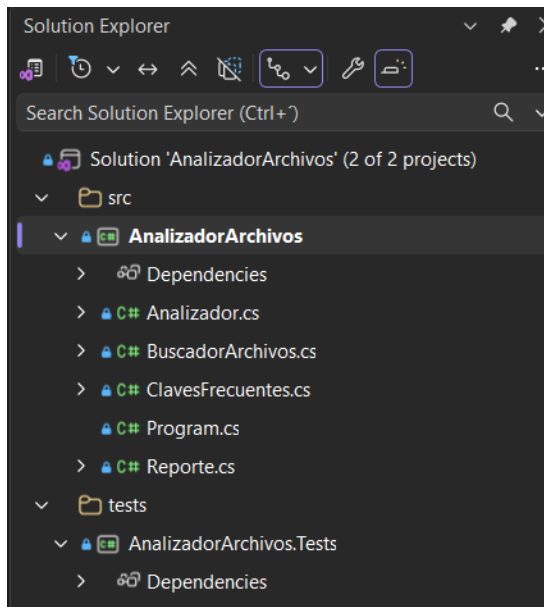
Manual de ejecución:

Requisitos: NET SDK 9.0 - NET SDK 10.0.

Se abre AnalizadorArchivos.sln en Visual Studio, se elige Release y se ejecuta con Ctrl+F5. La ruta de la carpeta a analizar se indica en la variable carpeta al inicio de src/Program.cs. El programa recorre esa carpeta y sus subcarpetas; si no existe o no contiene archivos .txt, muestra un aviso y no analiza nada.

Repositorio: <https://github.com/AnthonyGzm/analizador-archivos>

Capturas de pruebas complementarias



```
C:\Users\maria\OneDrive\Des  X  +  v

-----
Archivos analizados: 400
Total de lineas:      333,200
Total de palabras:    4,000,000
Total de caracteres:  22,205,398
-----
Coincidencias de terminos clave: 3,213,334
-----
Top 3 palabras mas repetidas:
  the -> 212,375
  and -> 114,277
  of  -> 110,071
-----
----Analisis de Rendimiento----
-----
Nucleos disponibles: 12

Modo Secuencial:      7,571.5 ms  (speedup 1.00x, eficiencia 100%)
Modo Paralelo x archivo: 2,879.7 ms (speedup 2.63x, eficiencia 22%)
Modo Paralelo x grupos: 2,777.1 ms (speedup 2.73x, eficiencia 23%)
-----
Verificacion:
-----
Secuencial:  4,000,000 palabras
Por archivo: 4,000,000 palabras
Por grupos:  4,000,000 palabras
Coincidencias: 3,213,334 = 3,213,334 = 3,213,334

Presiona ENTER para salir...
|
```